

Train Our First Linear Regression Model

```
[105]: from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train_processed, y_train)
```

```
y_pred = model.predict(X_test_processed)
```

```
[106]: print("First 10 predictions:")
```

```
print(y_pred[:10])
```

```
First 10 predictions:
```

```
[ 61874.25460143 121853.52511139 267770.94368091 264468.29837981
 258485.69585513 151961.4134933 308827.012159 244048.23081506
 258044.70385212 410870.2966254 ]
```

```
[107]: print("\nFirst 10 actual values:")
```

```
print(y_test.iloc[:10].values)
```

```
First 10 actual values:
```

```
[ 47700. 45800. 500001. 218600. 278000. 158700. 198200. 157500. 340000.
 446600.]
```

🔥 And notice something fascinating

Look at observation #3:

Prediction → 121,854 Actual → 500,001

The model missed by a huge amount.

But observation #5:

Prediction → 264,468 Actual → 278,000

is pretty close.

So we can't judge the model from one prediction.

We need a metric that summarizes the errors across all 4,128 test houses.

That's where:

MAE MSE RMSE R^2

come in.

Model Evaluation

```
[108]: from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, y_pred)

print("MAE:", mae)

MAE: 50898.7395349408
```

```
[110]: # Next: MSE – Mean Squared Error
```

```
[111]: from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)

print("MSE:", mse)

MSE: 5285482803.266442
```

▼ RMSE

```
[112]: from sklearn.metrics import root_mean_squared_error

rmse = root_mean_squared_error(y_test, y_pred)

print("RMSE:", rmse)

RMSE: 72701.32600762136
```

Now the final major metric: R^2

```
[114]: from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)

print("R²:", r2)

R²: 0.5966539537632211
```

✓ Approximately 59.7% of the variance in median_house_value is explained by the model on the test set.

Calculate the residuals

```
[115]: residuals = y_test - y_pred
```

```
[116]: print(residuals.iloc[:10])
```

```
20046    -14174.254601
3024     -76053.525111
15663     232230.056319
20484    -45868.298380
9814      19514.304145
13311      6738.586507
7113    -110627.012159
7668     -86548.230815
18246      81955.296148
5723       35729.703375
Name: median_house_value, dtype: float64
```

```
[118]: print("\nResidual statistics:")
print(residuals.describe())
```

```
Residual statistics:
count    4.128000e+03
mean     -4.787646e+02
std       7.270856e+04
min      -1.028753e+06
25%      -4.244981e+04
50%      -9.222392e+03
75%       3.013369e+04
max       4.533636e+05
Name: median_house_value, dtype: float64
```

```
[119]: import numpy as np
```

```
absolute_errors = np.abs(residuals)

largest_errors = absolute_errors.sort_values(
    ascending=False
).head(10)

print("\n10 largest absolute errors:")
print(largest_errors)
```

```
10 largest absolute errors:
2826    1.028753e+06
9877     4.999871e+05
6688     4.533636e+05
1979     4.141516e+05
10574    3.896654e+05
19542    3.775254e+05
12069    3.551350e+05
12389    3.497985e+05
13890    3.494979e+05
459      3.487626e+05
Name: median_house_value, dtype: float64
```

Model